

AI RESUME SCORER

Intelligent Resume Analysis & Job-Match Platform

Project Documentation | Version 1.0

Full-Stack Web Application | React.js + Node.js + MongoDB + AI/ML

INTRODUCTION

AI Resume Scorer is a powerful, intelligent web application designed to streamline and modernize the hiring process for both job seekers and recruiters. By leveraging cutting-edge Artificial Intelligence and Natural Language Processing (NLP) technologies, the platform automatically analyzes resumes, evaluates them against specific job descriptions, and provides a detailed compatibility score along with personalized improvement suggestions.

The platform offers an intuitive drag-and-drop resume upload interface, supports multiple file formats (PDF, DOCX, TXT), and provides instant, data-driven feedback. Recruiters can post job descriptions and receive ranked candidate lists, while applicants can benchmark their resumes against target roles and receive actionable tips to increase their chances of shortlisting.

AI Resume Scorer integrates seamlessly with HR tools, supports keyword extraction, ATS (Applicant Tracking System) compatibility checks, skill gap analysis, and readability scoring. Built-in data visualization dashboards allow HR teams to generate reports, track candidate pipelines, and export insights. Its flexibility, accuracy, and ease of use make it the ideal solution for recruitment agencies, corporate HR departments, career coaches, and individual job seekers.

SCENARIO BASED CASE STUDY

Meet Priya, a final-year Computer Science student who is actively applying for software developer roles at top tech companies. Despite having solid technical skills, Priya's job applications are not receiving responses, and she suspects her resume may not be optimized for Applicant Tracking Systems (ATS).

Priya discovers AI Resume Scorer, a comprehensive web application designed to bridge the gap between job seekers and their dream roles. Intrigued by its AI-powered features, she decides to explore how the platform can help her enhance her resume and improve her chances of landing interviews.

Priya's Journey on AI Resume Scorer

User Registration and Authentication

Priya registers an account on AI Resume Scorer, providing her professional details and preferences. She logs in securely using her credentials with JWT-based authentication, ensuring that her data remains private and protected.

Resume Upload and Parsing

Priya uploads her resume in PDF format. The platform's AI parser automatically extracts key information including her education, work experience, skills, certifications, and contact details. She can preview the parsed data and make corrections if needed.

Job Description Matching

Priya pastes the job description for a 'Full Stack Developer' role at a target company. The AI engine analyzes both documents, performs keyword matching, semantic similarity analysis, and skill alignment checks to generate a comprehensive match score.

Detailed Score Report

The platform provides Priya with a score of 62/100, broken down into categories: Keyword Match (70%), Skill Alignment (60%), ATS Compatibility (55%), Readability (80%), and Experience Relevance (65%). Each category includes specific recommendations for improvement.

Skill Gap Analysis

AI Resume Scorer identifies missing skills from the job description — such as Docker, Kubernetes, and GraphQL — and recommends online courses and certifications to bridge these gaps, linking directly to learning platforms.

Resume Enhancement Suggestions

The platform provides section-by-section suggestions: stronger action verbs for experience bullets, quantifiable achievements, improved summary statement, and proper formatting recommendations to boost ATS compatibility.

Progress Tracking and Dashboard

As Priya iterates on her resume, she can re-upload and track her score improvements over time through an interactive dashboard. She can also compare her resume across multiple job roles to find the best fit.

Priya's Outcome

After three iterations guided by AI Resume Scorer's recommendations, Priya's score improves to 87/100. She starts receiving interview calls within two weeks. AI Resume Scorer has become an indispensable tool in her job search, helping her present her skills with confidence and clarity.

SYSTEM REQUIREMENTS

To ensure smooth development, deployment, and usage of the AI Resume Scorer Web Application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications.

1. Software Requirements

Essential tools and platforms required to develop, test, and run the application efficiently:

Operating System	Windows 10/11, macOS 12+, or Linux (Ubuntu 20.04+) — Supports cross-platform development and testing
Node.js	v18 or above — Provides the runtime environment for building and managing frontend and backend logic, and powers API routing
npm	v9 or above — Package manager required to install dependencies for React, Express, and AI/ML libraries
React.js	v18+ — JavaScript library for building dynamic, responsive, and interactive user interfaces
Express.js	v4.18+ — Lightweight web framework for building RESTful APIs and handling server-side logic
MongoDB	v6+ — NoSQL database for storing user profiles, resumes, job descriptions, scores, and reports
Python 3.10+	Required for AI/NLP processing modules — spaCy, NLTK, scikit-learn, and PyMuPDF for PDF parsing
Browser	Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time
Postman	Tool for testing and debugging RESTful APIs during development
Visual Studio Code	Preferred code editor with built-in Git, terminal support, and extensions for JavaScript and Python
Docker (Optional)	For containerized deployment of backend services and AI microservices

2. Hardware Requirements

Minimum and recommended specifications to support development and runtime usage:

Processor	Intel Core i5 (10th Gen or above) / AMD Ryzen 5 or better — Ensures fast AI model inference and multitasking during development
RAM	Minimum 8 GB (16 GB recommended) — For running AI/NLP models, development servers, IDEs, and browser testing simultaneously

Storage	At least 5 GB free space — Required for AI model files, package installations, MongoDB setup, and local project files
GPU (Optional)	NVIDIA GPU with CUDA support — Recommended for faster AI model training and inference (not required for deployment)
Display	1366x768 or higher — Recommended for optimal coding experience and dashboard visualization
Internet	Stable broadband connection — Required for AI API calls, npm package installation, and cloud deployments

PROJECT ARCHITECTURE

The AI Resume Scorer follows a microservices-inspired layered architecture, separating the AI processing engine from the core web application backend to enable scalability and independent deployment.

High-Level Architecture Overview

Presentation Layer	Application Layer	Data & AI Layer
React.js Frontend User Dashboard Recruiter Portal Admin Panel	Node.js + Express REST API Gateway Auth Service File Upload Handler	MongoDB Database Python AI Engine NLP Processor File Storage

TECHNICAL ARCHITECTURE

The technical architecture of AI Resume Scorer is designed as a decoupled, API-driven system where the React.js frontend communicates with the Node.js backend via RESTful APIs, while a dedicated Python-based AI microservice handles all NLP and scoring computations.

Architecture Flow Description

The flow begins with the User (Job Seeker or Recruiter) interacting with the React.js Frontend. The user uploads a resume or submits a job description through the responsive web interface.

The React.js frontend sends HTTP requests to the Node.js / Express.js API Gateway, which acts as the central controller. The API Gateway validates requests, manages authentication via JWT tokens, and routes requests to appropriate service handlers.

For resume parsing and scoring tasks, the Node.js backend forwards the document to the Python AI Microservice via an internal REST API. The AI service leverages spaCy and NLTK for NLP processing, scikit-learn for similarity scoring, and PyMuPDF for PDF text extraction.

Processed results — including match scores, keyword analysis, skill gaps, and recommendations — are returned to the Node.js backend, which persists them in the MongoDB database and sends a structured JSON response back to the React frontend for rendering.

File uploads (resumes and supporting documents) are stored securely in a File Storage system (local or cloud-based such as AWS S3 or Cloundinary), with references stored in MongoDB.

Component Communication Table

From	To	Protocol / Method
User Browser	React.js Frontend	HTTP / HTTPS
React.js Frontend	Node.js API Gateway	REST API (JSON)
Node.js API Gateway	Python AI Microservice	Internal REST API (HTTP)
Node.js API Gateway	MongoDB Database	Mongoose ODM
Node.js API Gateway	File Storage	Multer / AWS SDK
Python AI Service	NLP Models (spaCy)	In-process library calls

ER DIAGRAM

The Entity-Relationship (ER) diagram for AI Resume Scorer represents the key entities and their relationships within the platform. The diagram captures data flow from user registration through resume submission, AI scoring, and recruiter evaluation.

1. Entities and Their Attributes

Feature	Description
User	userId (PK), name, email, password, role (jobseeker/recruiter/admin), createdAt, profilePicture
Resume	resumeld (PK), userId (FK), fileName, fileURL, uploadDate, parsedText, lastUpdated
JobDescription	jobId (PK), recruiterId (FK), title, company, description, requiredSkills, experienceLevel, postedDate
ScoreReport	reportId (PK), resumeld (FK), jobId (FK), totalScore, keywordScore, skillScore, atsScore, readabilityScore, experienceScore, generatedAt
SkillGap	gapId (PK), reportId (FK), missingSkill, suggestedCourse, priority
Suggestion	suggId (PK), reportId (FK), section, suggestionText, improvementType
Application	appld (PK), userId (FK), jobId (FK), reportId (FK), status, appliedAt
Admin	adminId (PK), name, email, password, permissions, lastLogin

2. Key Relationships

- User — Resume: One-to-Many (a user can upload multiple resume versions)
- User — Application: One-to-Many (a user can apply to multiple jobs)
- JobDescription — Application: One-to-Many (a job can receive multiple applications)
- Resume — ScoreReport: One-to-Many (a resume can be scored against multiple job descriptions)
- JobDescription — ScoreReport: One-to-Many (a job can generate scores for multiple resumes)
- ScoreReport — SkillGap: One-to-Many (each score report identifies multiple skill gaps)
- ScoreReport — Suggestion: One-to-Many (each report generates multiple improvement suggestions)

3. Primary and Foreign Keys

Each entity carries a unique primary key (e.g., `userId`, `resumeId`, `jobId`). Foreign keys establish relationships — for instance, `ScoreReport` holds both `resumeId` and `jobId` as foreign keys, linking the scoring outcome back to the specific resume and job description being compared.

KEY FEATURES

AI Resume Scorer offers a rich feature set designed to serve job seekers, recruiters, and administrators with intelligent, data-driven tools:

Feature	Description
AI Resume Parsing	Automatically extracts structured information (skills, education, experience, certifications) from uploaded PDF/DOCX/TXT resumes using NLP.
Job Description Matching	Compares resume content against a job description using semantic similarity, keyword frequency analysis, and skill alignment algorithms to produce a match score.
ATS Compatibility Check	Evaluates resume formatting, font usage, section headings, and keyword density against ATS standards, flagging potential parsing issues.
Skill Gap Analysis	Identifies skills present in the job description but absent from the resume, and suggests targeted learning resources to bridge the gap.
Improvement Suggestions	Provides actionable, section-by-section recommendations — stronger action verbs, quantifiable metrics, improved phrasing — to enhance resume quality.
Score Breakdown Dashboard	Displays a visual breakdown of the total score (0–100) across five dimensions: Keyword Match, Skill Alignment, ATS Compatibility, Readability, and Experience Relevance.
Multi-Resume Management	Users can store, manage, and compare multiple resume versions, tracking score improvements across iterations.
Recruiter Portal	Allows recruiters to post job descriptions, view ranked candidate shortlists, filter by score threshold, and download candidate reports.
Progress Tracker	Visual timeline showing score evolution across resume versions, motivating users to continuously improve.
Secure Authentication	JWT-based authentication with role-based access control (RBAC) for Job Seekers, Recruiters, and Administrators.
Bulk Resume Screening	Recruiters can upload multiple resumes at once for batch scoring against a single job description, streamlining high-volume hiring.
Export & Reporting	Generate and download PDF/Excel reports of score analyses, candidate rankings, and hiring pipeline summaries.

ROLES AND RESPONSIBILITIES

1. Job Seeker (User)

- **Registration and Profile Management**
 - Create an account, complete professional profile, and manage account security settings.
- **Resume Upload and Management**
 - Upload resume in PDF, DOCX, or TXT format. Manage multiple resume versions and track iteration history.
- **Job Description Submission**
 - Paste or upload a job description to initiate AI-powered scoring. Select target roles from integrated job board listings.
- **Score Review and Improvement**
 - Review detailed score reports, understand skill gaps, implement improvement suggestions, and re-score updated resumes.
- **Application Tracking**
 - Track application statuses, bookmark preferred roles, and monitor recruiter interactions through the dashboard.
- **Feedback and Ratings**
 - Provide feedback on AI suggestions and rate the platform experience to help improve recommendation quality.

2. Recruiter

- **Job Description Management**
 - Post, edit, and manage job descriptions. Set required skills, experience levels, and scoring thresholds for candidate filtering.
- **Candidate Screening**
 - View AI-ranked candidate lists for each job posting. Filter candidates by score, skill set, or experience level.
- **Bulk Resume Processing**
 - Upload multiple candidate resumes for batch scoring against a job description. Export ranked results as CSV or PDF reports.
- **Interview Pipeline Management**
 - Move candidates through hiring stages (Applied, Shortlisted, Interview Scheduled, Offer Made, Rejected) within the recruiter dashboard.

3. Administrator

- **System Management**
 - Monitor platform performance, manage server configurations, oversee AI model updates, and ensure data security and privacy compliance (GDPR, CCPA).
- **User Management**

- Review, approve, suspend, or delete user and recruiter accounts. Handle account disputes and policy violations.
- **AI Model Oversight**
 - Monitor AI scoring accuracy, review flagged reports, trigger model retraining when score quality degrades, and manage model versioning.
- **Analytics and Reporting**
 - Access platform-wide analytics: total resumes processed, average scores by industry, popular skill gaps, and user growth trends.

USER FLOW

Job Seeker Flow

Step 1: Register / Login — Create account or log in with existing credentials (JWT authentication).

Step 2: Upload Resume — Upload resume file (PDF/DOCX/TXT). AI parser extracts structured content automatically.

Step 3: Submit Job Description — Paste target job description or select from integrated job listings.

Step 4: Receive AI Score — View total match score (0-100) with breakdown across 5 scoring dimensions.

Step 5: Review Gaps & Tips — Analyze skill gaps, keyword mismatches, and section-wise improvement suggestions.

Step 6: Improve & Re-score — Update resume based on recommendations, re-upload, and track score improvement.

Step 7: Apply — Apply to jobs directly through the platform or export optimized resume for external use.

Recruiter Flow

Step 1: Register / Login — Create recruiter account with company verification and role-based access setup.

Step 2: Post Job Description — Create and publish a detailed job description with required skills, experience, and scoring thresholds.

Step 3: Receive Applications — Candidates apply through the platform; AI auto-scores each applicant's resume against the JD.

Step 4: Review Ranked List — View AI-ranked candidate shortlist sorted by match score, with skill-fit summaries for each applicant.

Step 5: Screen and Filter — Apply additional filters (min. score, specific skills, experience level) to narrow the candidate pool.

Step 6: Manage Pipeline — Move shortlisted candidates through interview stages within the recruiter dashboard.

Step 7: Export Reports — Download candidate comparison reports, score breakdowns, and hiring pipeline summaries.

MVC PATTERN

The AI Resume Scorer application follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates the application into three interconnected layers. This separation enables modularity, easier maintenance, independent testing, and horizontal scalability — critical requirements for an AI-powered application handling concurrent resume processing requests.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic, including the definition of data schemas and all database operations. Models are implemented using Mongoose, which provides a schema-based solution for MongoDB. The AI scoring results, resume metadata, and user profiles are all defined and managed at this layer.

- Users.js — Schema for job seeker and recruiter accounts
- Resume.js — Schema for uploaded resumes and parsed content
- JobDescription.js — Schema for recruiter-posted job descriptions
- ScoreReport.js — Schema for AI-generated scoring results
- SkillGap.js — Schema for identified skill gaps and suggestions
- Application.js — Schema for job applications and pipeline stages

Controller Layer

The Controller layer acts as an intermediary between the View (routing layer) and the Model. It receives incoming API requests, validates and transforms input, invokes the appropriate Model methods, coordinates with the Python AI microservice for scoring operations, and returns structured JSON responses to the client.

- authController.js — Handles registration, login, JWT token generation
- resumeController.js — Manages resume upload, parsing, version control
- scoringController.js — Orchestrates AI scoring pipeline and result storage
- jobController.js — Handles JD creation, management, and application routing
- recruiterController.js — Powers recruiter dashboard, candidate ranking, pipeline
- adminController.js — System management, user oversight, analytics

View Layer (Routing Layer)

In the context of this REST API backend, the View is implemented as the routing layer where API endpoints are defined. These routes determine how the server responds to different HTTP methods (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions. The actual visual rendering is handled entirely by the React.js frontend.

Advantages of MVC in AI Resume Scorer

Separation of Concerns	AI scoring logic, API routing, and UI rendering are fully decoupled — changes in one layer don't affect others.
Scalability	The Python AI microservice can be scaled independently from the Node.js API when scoring demand increases.
Testability	Controllers and models can be unit-tested independently; the AI engine can be evaluated with benchmark resume datasets.
Reusability	Scoring logic in controllers can serve both the web interface and future mobile applications or API clients.
Collaboration	Frontend developers, backend engineers, and ML engineers can work simultaneously without conflicts.

PROJECT SETUP AND CONFIGURATION

1. Creating Project Folder Structure

1. Create a new root folder named ai-resume-scorer/
2. Inside it, create three sub-folders: client/, server/, and ai-engine/
3. The client/ folder holds the React.js frontend
4. The server/ folder holds the Node.js + Express backend
5. The ai-engine/ folder holds the Python NLP/AI microservice
6. Open the root folder in VS Code

2. Client Setup (React + Vite)

- Open the client/ folder in the VS Code terminal
 - Run: `npm create vite@latest . -- --template react`
- Select React as the framework and JavaScript as the variant
- Install all dependencies: `npm install`
- Install additional required libraries:
 - `npm install axios react-router-dom recharts react-dropzone`
 - `npm install react-circular-progressbar react-hot-toast`
- Start the development server: `npm run dev`
- The app will run on `http://localhost:5173`

3. Server Setup (Node.js + Express)

- Open the server/ folder in the VS Code terminal
 - Run: `npm init -y`
- Install required packages:
 - `npm install express mongoose dotenv cors jsonwebtoken bcryptjs multer`
 - `npm install axios nodemon`
- Create the following directory structure:
 - controllers/ — Business logic handlers
 - models/ — Mongoose schemas
 - routes/ — API route definitions
 - middleware/ — Auth, file upload, error handling
 - utils/ — Helper functions and validators
 - uploads/ — Temporary file storage
- Create server.js as the application entry point
- Configure .env with: PORT, MONGO_URI, JWT_SECRET, AI_ENGINE_URL
- Start the server: `npx nodemon server.js`

4. AI Engine Setup (Python)

- Open the ai-engine/ folder in the VS Code terminal
- Create a virtual environment: `python -m venv venv`
- Activate it and install dependencies:
 - `pip install flask spacy nltk scikit-learn PyMuPDF python-docx`
 - `python -m spacy download en_core_web_md`
- Create app.py as the Flask API entry point for NLP services
- Start the AI engine: `flask run --port 5001`

BACKEND DEVELOPMENT

Backend Structure

Feature	Description
controllers/authController.js	Handles user registration, login, JWT generation, password hashing with bcrypt, and token refresh logic.
controllers/resumeController.js	Manages resume file upload via Multer, triggers AI parsing, stores parsed content and file metadata in MongoDB.
controllers/scoringController.js	Orchestrates the AI scoring pipeline — sends resume text + JD to Python engine, receives scores, stores reports, returns breakdown to client.
controllers/jobController.js	Handles CRUD operations for job descriptions, manages application submissions, and links candidates to job postings.
controllers/recruiterController.js	Powers the recruiter dashboard — ranked candidate lists, pipeline stage updates, bulk resume processing, and report exports.
controllers/adminController.js	System management — user oversight, AI model version management, platform analytics, and content moderation.
middleware/authMiddleware.js	JWT token validation and role-based access control (RBAC) — differentiates between jobseeker, recruiter, and admin routes.
middleware/upload.js	Multer configuration for handling resume file uploads — validates file type (PDF, DOCX, TXT) and size limits (max 5MB).
models/Users.js	Mongoose schema for user accounts with role differentiation (jobseeker / recruiter / admin).
models/Resume.js	Schema for uploaded resumes, parsed text content, version history, and associated score reports.
models/ScoreReport.js	Schema for AI-generated scoring results with sub-scores for each evaluation dimension.
models/JobDescription.js	Schema for job postings with required skills array, experience level, and scoring threshold configuration.
routes/authRoutes.js	POST /register, POST /login, POST /refresh-token, POST /logout endpoints.
routes/resumeRoutes.js	POST /upload, GET /my-resumes, GET /resume/:id, DELETE /resume/:id endpoints.
routes/scoringRoutes.js	POST /score, GET /report/:id, GET /my-reports, GET /compare endpoints.
routes/recruiterRoutes.js	GET /candidates, PUT /pipeline/:appId, POST /bulk-score, GET /export endpoints.

DATABASE DEVELOPMENT

1. Configure MongoDB

- Install Mongoose in the server/ directory:
 - `npm install mongoose`
- Create the database connection file at `db/config.js`
- Store the MongoDB connection URI in the `.env` file as `MONGO_URI`
- Call `connectDB()` before starting the Express server in `server.js`

2. Schema Definitions

All Mongoose schemas are derived from the ER diagram entities. Below are the core schemas for the AI Resume Scorer application:

Users.js — User Account Schema

Fields: `name` (String), `email` (String, unique), `password` (String, hashed), `role` (String: enum ['jobseeker', 'recruiter', 'admin']), `profilePicture` (String), `createdAt` (Date, default: `Date.now`)

Resume.js — Resume Document Schema

Fields: `userId` (ObjectId, ref: 'User'), `fileName` (String), `fileURL` (String), `parsedText` (String), `skills` (Array), `education` (Array), `experience` (Array), `uploadDate` (Date), `versionNumber` (Number, default: 1)

ScoreReport.js — AI Score Report Schema

Fields: `resumeld` (ObjectId, ref: 'Resume'), `jobId` (ObjectId, ref: 'JobDescription'), `totalScore` (Number: 0-100), `keywordScore` (Number), `skillScore` (Number), `atsScore` (Number), `readabilityScore` (Number), `experienceScore` (Number), `skillGaps` (Array), `suggestions` (Array), `generatedAt` (Date, default: `Date.now`)

JobDescription.js — Job Description Schema

Fields: `recruiterId` (ObjectId, ref: 'User'), `title` (String), `company` (String), `description` (String), `requiredSkills` (Array), `experienceLevel` (String), `minimumScore` (Number), `postedDate` (Date, default: `Date.now`), `isActive` (Boolean, default: `true`)

Application.js — Job Application Schema

Fields: `userId` (ObjectId, ref: 'User'), `jobId` (ObjectId, ref: 'JobDescription'), `resumeld` (ObjectId, ref: 'Resume'), `reportId` (ObjectId, ref: 'ScoreReport'), `status` (String: enum ['applied', 'shortlisted', 'interview', 'offer', 'rejected']), `appliedAt` (Date, default: `Date.now`)

FRONT-END DEVELOPMENT

Frontend Structure

The frontend is organized into role-specific modules under `src/`, enabling clean separation of Job Seeker, Recruiter, and Admin interfaces with shared reusable components.

Admin Module (`src/Admin`)

Feature	Description
<code>AdminDashboard.jsx</code>	Main admin panel with platform-wide stats: total users, resumes processed, avg. score, active jobs.
<code>UserManagement.jsx</code>	View, search, suspend, or delete user and recruiter accounts. Role modification interface.
<code>ModelAnalytics.jsx</code>	Monitor AI scoring accuracy trends, score distribution histograms, and model performance metrics.
<code>ReportCenter.jsx</code>	Generate and export platform analytics reports as PDF or CSV.
<code>AdminLogin.jsx</code>	Secure admin authentication page with 2FA support.
<code>AdminNavbar.jsx</code>	Admin-specific navigation bar with quick access to all management modules.

Job Seeker Module (`src/User`)

Feature	Description
<code>UHome.jsx</code>	Job seeker homepage with quick-upload widget, recent score history, and personalized job recommendations.
<code>ResumeUpload.jsx</code>	Drag-and-drop resume upload interface with file validation, preview, and version labeling.
<code>ScoreResult.jsx</code>	Displays AI score breakdown with animated circular progress bars for each scoring dimension.
<code>SkillGapView.jsx</code>	Visual skill gap analysis with missing skills highlighted and linked learning resources.
<code>SuggestionPanel.jsx</code>	Section-by-section improvement suggestions with before/after preview for key resume sections.
<code>MyResumes.jsx</code>	Resume version history with score comparison chart across iterations.
<code>ProgressTracker.jsx</code>	Timeline chart showing score evolution from first upload to latest version.

UNavbar.jsx	Job seeker navigation bar with links to dashboard, resumes, applications, and profile settings.
ULogin.jsx / USignup.jsx	User authentication pages with form validation and social login options.

Recruiter Module (src/Recruiter)

Feature	Description
RecruiterDashboard.jsx	Recruiter homepage with active job stats, recent applications count, and shortlist alerts.
PostJob.jsx	Job description creation form with required skills tag input, experience level selector, and scoring threshold slider.
CandidateList.jsx	AI-ranked candidate list with score badges, skill-fit indicators, and quick-view resume panels.
BulkUpload.jsx	Multi-file resume upload interface for batch scoring against a selected job description.
PipelineBoard.jsx	Kanban-style hiring pipeline board (Applied → Shortlisted → Interview → Offer → Hired).
ExportReport.jsx	Generate downloadable PDF/CSV reports of candidate rankings and score breakdowns.

Shared Components (src/Components)

Feature	Description
Navbar.jsx	Responsive navigation bar adapting to user role context.
Footer.jsx	Application footer with links to documentation, support, and legal pages.
ScoreCard.jsx	Reusable score display card with circular progress indicator and color-coded bands.
SkillBadge.jsx	Reusable chip component for displaying matched and missing skills with color coding.
FileDropzone.jsx	Shared drag-and-drop file upload component with validation feedback.
LoadingSpinner.jsx	Animated loading indicator for AI processing wait states.
ui/	Shared UI primitives: Button, Modal, Toast, Tooltip, Badge, Tabs, Card components.

AI ENGINE (PYTHON MICROSERVICE)

The Python AI microservice is the intelligence core of AI Resume Scorer. It exposes a Flask REST API consumed exclusively by the Node.js backend. All computationally intensive NLP and ML operations are isolated in this service for independent scaling and maintenance.

Key AI/NLP Components

Feature	Description
Resume Parser (parser.py)	Uses PyMuPDF (fitz) for PDF text extraction and python-docx for DOCX parsing. Normalizes whitespace, removes headers/footers, and segments text into logical resume sections.
Keyword Extractor (keyword.py)	Extracts technical keywords, tools, and technologies from both resume and JD using spaCy's NER and a custom tech-skills vocabulary of 3,000+ terms.
Semantic Scorer (similarity.py)	Computes TF-IDF vectors and cosine similarity between resume content and JD text using scikit-learn. Also applies spaCy's word vectors for semantic (contextual) matching beyond keyword overlap.
ATS Checker (ats.py)	Evaluates resume structure against ATS parsing rules: standard section headings, single-column layout indicators, avoidance of tables/graphics in text flow, and proper date formatting.
Readability Analyzer (readability.py)	Computes Flesch-Kincaid readability score, average sentence length, and action verb density for the professional summary and experience sections.
Skill Gap Engine (gaps.py)	Compares extracted skills from the resume against the JD's required skills list. Classifies missing skills by priority (Critical / Recommended / Nice-to-have) and maps each to a curated learning resource.
Suggestion Generator (suggestions.py)	Rule-based and template-driven suggestion engine that generates context-aware improvement tips per resume section based on identified weaknesses.
API Gateway (app.py)	Flask application exposing POST /parse, POST /score, POST /ats-check endpoints consumed by the Node.js backend.

CONCLUSION

AI Resume Scorer represents a significant advancement in intelligent recruitment technology. By combining the power of Natural Language Processing, semantic similarity analysis, and ATS compatibility evaluation within an intuitive full-stack web application, the platform delivers meaningful value to both sides of the hiring equation.

For job seekers like Priya, the platform transforms the often-opaque resume optimization process into a data-driven, iterative improvement cycle with clear, actionable feedback. For recruiters, it replaces time-consuming manual resume screening with an AI-powered shortlisting system that surfaces the most qualified candidates instantly.

The application's architecture — built on React.js, Node.js/Express, MongoDB, and a Python AI microservice — ensures scalability, maintainability, and the flexibility to incorporate increasingly sophisticated AI models as the field of NLP continues to advance. The MVC pattern, clean separation of concerns, and modular component design make it straightforward for development teams to extend the platform with new features such as video resume analysis, LinkedIn integration, or real-time interview coaching.

AI Resume Scorer is not just a tool — it is a career accelerator and a hiring optimizer, engineered to make the right connections between talent and opportunity with the precision and consistency that only AI can provide.

Link for demo project:-  fullstack-projectdemo.mov

Link for code explanation:-  fullstack-projectexplanation.mov